

Disengagement in Pair Programming: Does It Matter?

Laura Plonka
Centre for Research in Computing
The Open University
Milton Keynes, United Kingdom
l.f.plonka@open.ac.uk

Helen Sharp
Centre for Research in Computing
The Open University
Milton Keynes, United Kingdom
h.c.sharp@open.ac.uk

Janet van der Linden
Centre for Research in Computing
The Open University
Milton Keynes, United Kingdom
j.vanderlinden@open.ac.uk

Abstract—Pair Programming (PP) requires close collaboration and mutual engagement. Most existing empirical studies of PP do not focus on developers' behaviour during PP sessions, and focus instead on the effects of PP such as productivity. However, disengagement, where a developer is not focusing on solving the task or understanding the problem and allows their partner to work by themselves, can hinder collaboration between developers and have a negative effect on their performance. This paper reports on an empirical study that investigates disengagement. Twenty-one industrial pair programming sessions were video and audio recorded and qualitatively analysed to investigate circumstances that lead to disengagement. We identified five reasons for disengagement: interruptions during the collaboration, the way the work is divided, the simplicity of the task involved, social pressure on inexperienced pair programmers, and time pressure. Our findings suggest that disengagement is sometimes acceptable and agreed upon between the developers in order to speed up problem solving. However, we also found episodes of disengagement where developers “drop out” of their PP sessions and are not able to follow their partner's work nor contribute to the task at hand, thus losing the expected benefits of pairing. Analysis of sessions conducted under similar circumstances but where mutual engagement was sustained identified three behaviours that help to maintain engagement: encouraging the novice to drive, verbalisation and feedback, and asking for clarification.

Keywords—collaboration; agile software development; empirical study

I. INTRODUCTION

After ten years, exploring agile software development is still of special interest to researchers and practitioners. For example, this year, the UK government decided to adopt agile practices across all its Departments [1]. One of the controversial key practices of agile software development is pair programming (PP), where two developers sit together to solve one problem. A wide range of studies [2], [3], [4], [5], [6] has investigated PP; most of the studies focus on the effects and report positive results indicating benefits of PP. However, some studies are inconclusive and some report negative results (a meta analysis of 18 PP studies can be found in [7]). This indicates the need for studies investigating factors that influence the effects of PP. External factors such as task complexity [8] and the developers' personality [9] have been found to influence the effects of

PP, but little emphasis has been placed on what actually happens within the PP session.

Williams and Kessler [10] attributed seven synergistic behaviours to the success of PP and Wray [11] suggested four mechanisms that “make PP really work” based on his industrial experience. Those behaviours and mechanisms include pair negotiation which refers to the process of pair brainstorming and results in highly effective problem solving, pair reviews which use the concept of “four eyeballs are better than two” and lead to early fault detection, and pair learning which refers to a constant sharing of knowledge among the developers and leads to effective learning in pairs.

In all cases, these behaviours require both developers to actively collaborate and to be engaged in the problem-solving activity. We define collaboration as “mutual engagement of participants in a coordinated effort to solve a problem together” [12]. Williams et al. [10] stress the role of collaboration in PP by stating that “pair programming works when the pairs are tightly integrated, interacting and working closely”.

In this paper, we focus on disengagement of pair programmers in order to examine the details of pairing sessions. We define disengagement as one developer not focussing on solving the task or understanding the problem and allowing their partner to work by themselves. Disengagement is a potential problem in developers' collaboration that hinders the behaviours and mechanisms that positively influence the effects of PP. Here, we present an exploratory study that is based on video and audio recordings of twenty-one industrial PP sessions and post-session interviews with pairs of developers. We use a qualitative analysis approach because prior quantitative research on developers' interaction in PP has shown that a quantitative perspective provides only limited insights into the details of pairing sessions [13]. As a result of our qualitative analysis, we present a series of circumstances and behaviours that are related to disengagement and contrast this with behaviours exhibited in sessions in which both developers were mutually engaged throughout the whole session. We identified strategies to avoid episodes of disengagement and our findings suggest that disengagement is not always an “undesirable” behaviour, but can be a negotiated strategy agreed on by both developers.

II. RELATED WORK

A. Collaboration and Mutual Engagement

Collaboration can enable groups to achieve tasks that individual members could not achieve on their own [14].

According to Roschelle and Teasley [12], collaboration is “mutual engagement of participants in a coordinated effort to solve a problem together” [12]. Miell and MacDonald [15] state that mutual engagement is indicated by the “presence of reasoned dialogue, the exploration of the ideas of more than one person and the attempt to integrate these”. In the context of creative collaboration, Bryan-Kinns et al. [16] suggest that mutual engagement may be indicated by features of interactions such as contributions and modification of contributions to the joint product and acknowledgement, repetition and modification of others’ contributions.

B. Collaboration in Pair Programming

Most of the existing empirical studies focus on the effects of PP without considering the nature of collaboration between the developers. In this section, we present an overview of those studies that do focus on the collaboration of professional developers in PP sessions.

1) *Collaboration and Personality*: Walle and Hanay [17] studied the effect of personality on pair collaboration based on audio-recordings of 44 PP sessions. They postulated and tested six different relationships between personality and collaboration. Their results did not support very specific relationships but they found more generally that personality affects collaboration in PP sessions and that developers with different levels of certain personality traits have a higher amount of communication-intensive collaboration. For example, developers exhibit cross purpose interaction patterns where developers are speaking on separate topics or responsive interaction patterns where questions and responses of both developers contribute substantive statements to the discussion.

2) *Collaboration and Programmers’ Experience and Expertise*: Bryant et al. [18] focused on the collaboration between pair programmers who had a minimum of six months of commercial pair programming experience. Using audio recordings from four one-week studies they focused on the contributions each developer made to the task at hand. They found that most of the contributions refer to a subtask that Bryant et al. call “understanding the problem or the existing code”, followed by “writing new code” and “testing”. Additionally, they concluded that PP is highly collaborative because both developers contribute to almost all subtasks and the contributions of the “driver”, that is the developer who is in control of mouse and keyboard, and the “navigator”, i.e. the other developer, were almost equally distributed. The developers can switch between the roles of driver and navigator at any time. In another study, Bryant [19] investigated the types of interaction developers

entered into and also how often they did so. They found that novice pair programmers interact more than expert pair programmers and that novice pair programmers “suggest” and “counter-suggest” more frequently than experts. Novice pair programmers also entered into different types of interaction depending on whether they were in the role of driver or navigator. In contrast, the expert pair programmers did not show a change in behaviour whether they were driving or not.

Chong and Hurlbutt [20] also focused on the impact of different levels of expertise on developers’ interaction. They found that the distribution of expertise in development teams influences the developers’ collaboration in PP sessions. In a team with uniform distribution, developers with more expertise tried to achieve a shared level of expertise with their partner and then “proceed to pair normally” whereas in a team with big differences in expertise among the developers, developers with more expertise tend to dominate the interaction and tend to give the less experienced developers instructions.

Rostaher and Hericko [21] investigated the effect of programming experience on a specific aspect of interaction; the frequency of driver-navigator switches. They found that the frequency of changes correlates with programming experience and that more experienced pairs change most often.

3) *Collaboration and the Roles of Driver and Navigator*: Bryant et al. [22] investigated the interactions of driver and navigator further, focusing on the level of abstraction of the roles of driver and navigator. In contrast to the popular belief that the role of the driver is more practical and that the navigator works at a higher level of abstraction, they showed that a) driver and navigator tend to work on the same level of abstraction and b) that a significant amount of talk is at an *intermediate* level of abstraction. The first result is reported also by Chong and Hurlbutt [20], who stated that “aside from the task of typing” there was no consistent division of labour between the driver and navigator. Furthermore, they noticed that the developer who had control over the keyboard had more authority in decision making. Plonka et al. [23] also investigated the roles of driver and navigator. They found that developers do not evenly contribute to the task of driving, that most pairs switch roles frequently, and that the frequency and fluidity of switches indicate a high level of engagement on the part of both the developers. Moreover, their results show that developers spend on average a third of the session without any computer interaction, focusing instead on communication.

The studies summarised above provide insights into the collaboration of pair programmers and investigate potential factors that might influence the collaboration. However, they do not focus on whether and how different types of interaction in collaboration might influence pair performance. Moreover, most of the studies are based on notes that have been taken in real-time and audio-recordings.

Since collaboration between developers can be very rich, we base our investigations on video recordings. This allows the researchers to replay important episodes and to analyse interactions in more detail.

III. RESEARCH SITES

This paper is based on four one-week studies of industrial PP sessions from four different companies. The selection of the companies was opportunistic. Each company was a different size and all belonged to different industries (table I provides an overview of the companies). All four companies used agile approaches; two had just introduced PP and used Scrum, the other two had used PP for more than one year and used an adapted agile approach based on Scrum. All the software development teams that we observed were established development teams rather than project teams who work together only for a specific project. Each team was collocated, with the developers in the same office. PP was encouraged by the team leaders in all development teams but developers decided when to use PP and with whom to pair. The forming of the pairs was done during their daily meeting or spontaneously throughout the day. Table I provides an overview of the developers' background.

IV. DATA GATHERING

We used three types of data gathering: questionnaires, audio and video recordings, and interviews.

- 1) **Questionnaires** were used to gather background information about the developers: why they were pairing, information about the task for which they had chosen to work in a pair such as the type of task, and experience of the developers in programming and PP. Participants were asked to complete these before starting their pairing session.
- 2) **Audio and video recordings** were used to record the PP sessions. The recordings of the session consist of the following three data sources:
 - Audio recordings of the verbal communication between the participants,
 - a video of the programmers as they worked on the pairing task,
 - and a full-resolution screen recording capturing all computer activities of the programmers.

All three data sources are fully synchronized and stored in a single video file so that all information is available at once (see figure 1). The PP sessions were recorded in the developers' day-to-day work environment and while solving day-to-day tasks. Our recording setup was installed at one computer in each participating company (see figure 1).

- 3) **Interviews** were conducted with both developers present one day after the session to capture their account of the PP sessions. This allowed us to analyse the video identifying episodes of disengagement

Table I
OVERVIEW: COMPANIES' AND DEVELOPERS' BACKGROUND

Industry	Company size	Team size	Programming experience	PP experience
Geographic information systems	30-50	8	0.9-20 years	2-20 years
Traffic, logistic and transport	<500	two teams 5 developers each	0.4-13 years	0-3 years
Email marketing	50-100	8	1.3-10 years	0-5 years
Estate CRM Software	50-100	10	1.5-12 years	0-3.6 years

before the interviews and then to discuss potential episodes of disengagement with the developers.

Participation in our study was voluntary; in total 21 PP sessions with 31 developers were recorded, with some developers being recorded more than once. The developers worked in pairs with 20 different permutations of developers. There was only one repeated pair permutation. See Table II for an overview. A PP session lasted between 1.5 hours and 3.5 hours; in total we video taped about 37 hours of PP sessions. During our studies, the developers worked on their day-to-day tasks in their usual working environment. We

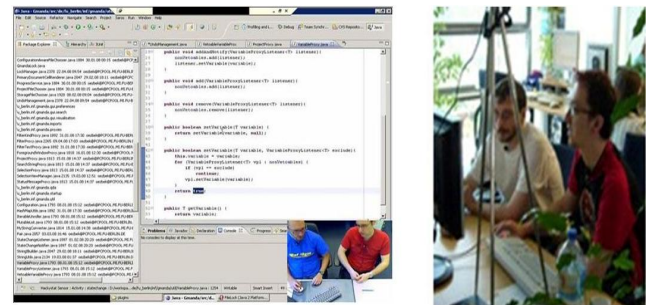


Figure 1. On the left: Screenshot of a fully synchronized video of a PP session: This screenshot shows the screen of the developers' computer (Eclipse IDE). On the bottom right, a screenshot of the video of the developers is shown. On the right: Recording setup in one of the companies. The setup was installed in the usual working environment of the developers.

did not interfere with their usual work practice by assigning pairs or tasks. The first author was present at the companies' sites during the data gathering week but was not located in the same office as the developers and was not present in the developers' office during the video recordings. During the week, we asked the developers to inform us when they wanted to start a PP session. At this point they were handed the questionnaires, and recordings were started. During the

Table II
OVERVIEW: NUMBER OF RECORDINGS

Company	# sessions	# developers	# different pair permutations
1	6	8	6
2	7	8	6
3	4	6	4
4	4	7	4
In total	21	31	20

recordings, the developers worked in their usual environment on the computer that was set up for the recordings.

V. ANALYSIS METHOD

Our analysis method consists of three steps: (1) identify episodes of disengagement in PP sessions, (2) investigate circumstances leading to disengagement, and (3) explore how disengagement in PP sessions can be avoided.

Our approach to identifying episodes of disengagement is based on indicators for mutual engagement. Therefore in this section, we begin with an explanation of the indicators of mutual engagement, followed by the three steps of our analysis which will be illustrated through examples.

A. Indicators for Mutual Engagement

We used indicators suggested by Bryan-Kinns et al. [16] and Miell and MacDonald [15] as starting points to identify mutual engagement in PP.

Bryan-Kinns et al. and Miell and MacDonald examined mutual engagement in the context of collaborative music making. In those studies, participants worked as pairs and were involved in a focused activity. In the study of Bryan-Kinns et al., participants worked remotely and had multiple input devices that could be used synchronously by the participants. Bryan-Kinns et al. suggested six indicators for mutual engagement for the context of their study, each of which represents a certain level of engagement, i.e. basic, medium, increased, high. We used those indicators as a starting point for our study, focusing in particular on “Contribution to the joint production”, “Acknowledgement”, “Mirroring”, and “Modification”. The other two indicators we ignore as they are not relevant for our study: “Proximal interaction”, as our participants are co-located rather than working remotely, and “Mutual modification” since our participants have a single device to share rather than multiple input devices.

Based on the suggestion by Miell and MacDonald [15] that mutual engagement is indicated by presence of reasoned dialogue, we extended the list of indicators and added a new indicator (Question/Responses) and to be consistent with the other indicators, we assigned it a “medium level” of engagement.

Based on the considerations above, we propose the following characteristics to identify episodes that indicate mutual engagement in PP:

- **Contribution to the joint production**

Developers are contributing to a common goal with both developers focusing on solving the task together. Developers do not divide the task in subtasks and do not work independently. Contributions in PP can be in the form of verbal contributions or driving contributions. Additionally, a developer can contribute by using external representations or by pointing on the screen, for example in order to answer a question. Contribution to the joint production indicates an increased level of mutual engagement.

- **Acknowledgement/Feedback**

Developers show an awareness of each other’s idea by acknowledging or giving feedback to an idea or an action (for example nodding, saying “mhm” or by pointing at something on the screen). This indicates a basic level of mutual engagement.

- **Mirroring**

Developers mirror a contribution of their partner, for example, when the navigator is verbalising what the driver is typing/doing or when developers repeat and reflect verbal contributions of their partner in their own words. Mirroring indicates a medium level of mutual engagement.

- **Modification**

A high level of mutual engagement is indicated by developers transforming or modifying the contributions of their partners.

- **Questions/Responses**

Aside from making direct contributions, developers can engage with their partners by asking for explanations or clarifications and by providing explanations. This can be either verbally or by showing the partner something on the screen or implementing code, for example, a method to show how the idea would be implemented. Questions and responses indicate a medium level of mutual engagement.

A short excerpt of a video recording is presented to illustrate an episode that indicates mutual engagement.

Peter and Mike are browsing the apache maven project documentation to find a solution for their current problem. Peter is driving.

Peter: “Jar:jar would create a jar with this name” <pointing with the mouse to the name>

Mike: “Mhm.”

Peter: “And the jar:deploy would be interesting, of course”

Mike: “Oh yes, that means if we use jar:jar with attainGoal, that could be it. That might do what we want already and jar:deploy would be the next step. I’d say, we try jar:jar first and see if it works.”

Peter: “Exactly, because this <pointing to jar:deploy> would then call jar:jar.”

Peter browses another website.

Mike: “Right, but wait I tried that already and. Hm, open the DOS window for a moment!”

Peter switches to the DOS window.

Peter starts typing: “That was maven”

Mike continues his sentence: “*maven/jar:deploy*” Peter is typing the command.

This episode is an example of mutual engagement. Both developers are engaged with the implementation of the solution; developers contribute to the same topic, they acknowledge contributions by their partners (“*hmh*”, “*exactly*”), they reflect and elaborate on them (for example, “*Oh yes, that means if we use jar:jar with attainGoal[...]*”).

In the following sections, we show how we used these indicators to investigate disengagement in PP.

B. Step 1: Identifying Episodes of Disengagement in PP Sessions

For our study, we describe *disengagement* as one developer not focussing on solving the task or understanding the problem and allowing their partner to work by themselves. In our analysis, we use the absence of mutual engagement as indicator for disengagement. This allowed us to identify episodes in the videos that indicate disengagement. Furthermore, for those episodes where developers show only a basic level of mutual engagement we decided to delve deeper (as described in step 2).

However, the absence of indicators for mutual engagement is based on observable behaviours and ignores the fact that developers might be actually *thinking* about the problem at hand and following their partners’ actions or explanations without making that visible. Therefore, we looked for confirmation by the developers that they were actually disengaged.

We used two types of confirmations to reassure that those episodes were real episodes of disengagement; (1) verbal confirmation by the developers during the PP session or (2) confirmation by the developers during the interviews.

(1) During the PP session, some developers made it verbally explicit to their partners that they did not know what their partner was working on and that they were not able to contribute to the task as illustrated in the following excerpt from a video:

Dave and Mark are working together on a task. Dave is driving and explaining to Mark a concept that is new to him. Mark does not reply or react to Dave’s explanation. Dave keeps explaining and driving. When Dave finishes his explanation he keeps driving in silence for a little while, then turns around to Mark and asks: “Do you want to drive?” Mark is shaking his head: “I’m not sure what you’re doing at the moment. I’m a bit lost here. So you continue driving and I let you know when I know again what’s going on.”

We interpret the statement from Mark as a confirmation of disengagement as his statement makes explicit that he was not focusing on the task at hand or following his partner.

(2) During the post-session interviews, developers gave their account of the PP session and could confirm or contradict whether they were disengaged or not. Here, we present two examples from the interviews:

Example: confirmation of disengagement by the developers during the post-session interview:

Richard: “*I need feedback from my partner and I don’t want Robin to just say yes. So I ask for feedback and it made me a bit nervous, ok that is an exaggeration, but I noticed that he stopped responding to my questions, even though I kept asking.*”

Robin: “*There were some situations where I was not sure what Richard was doing while he was driving.*”

Example: rejection of disengagement by the developers during the post-session interview:

“*It took me some time to reply but that was just because I was still thinking whether this is really the right way to do it.*”

The first excerpt shows an example of a confirmation of disengagement that was recognised by both developers. The second excerpt provides an example of a rejection of disengagement by the developer, i.e. although it appeared they were disengaged, in fact the developers were thinking about the task.

C. Step 2: Investigation of Circumstances Leading to Disengagement

The episodes of disengagement are investigated further by analysing the circumstances related to that episode. This provides insights into the circumstances when disengagement might happen which we analysed by asking:

- what happens in the PP session before the episodes of disengagement and in the episodes of disengagement (based on the videos),
- what is the background of the developers (based on the questionnaires),
- and how do developers reflect on their PP sessions (based on the interviews).

In some cases, the episodes in the videos and the interview data *both* provided insight into the reasons that led to disengagement. In other cases, the video recordings were not particularly insightful but the interviews were.

D. Step 3: Explore how Disengagement in PP Sessions Can Be Avoided

The third analysis step is informed by the results of the second step. We investigate the interactions between developers who did not face the issue of disengagement despite working under the same circumstances identified in the first step. To support this, we identify sessions which took place under these same circumstances (based on the questionnaires and interviews), and analysed the differences in behaviour between the session in which disengagement was exhibited and sessions in which it was not. The differences in behaviours are analysed based on the video data.

Example excerpts of video and interview data for the second and third step are provided in the next section to illustrate our findings.

VI. FINDINGS: CIRCUMSTANCES LEADING TO DISENGAGEMENT

We identified five different sets of circumstances that can lead to disengagement in PP sessions, which we describe here. All names in the excerpts have been replaced with pseudonyms and all excerpts have been edited to maintain confidentiality.

A. Interruptions

Based on video data, we observed that interruptions by other co-workers can lead to disengagement of developers as illustrated in the following video excerpt.

In this session, Sandra and Pete are working with a framework to solve a task about a special case of data entries. Neither of them is familiar with the framework.

Sandra is driving, she is verbalising some of her actions and Pete is pointing out what else should be considered and changed.
Pete: *"Here TraitViewFactory."*
Sandra keeps typing: *"Trait must be an interactive one here."*
Pete: *"Here? Hm, yes!"*
Sandra: *"We do not create interactive traits in other positions, so we can change it."*
Pete: *"Yes."*
Co-worker starts talking to Pete: *"I tested the interface again and it did not work [...]"*
Sandra continues driving without verbalising any of her steps.
Pete replies to the co-worker: *"I have no idea what you are talking about."*
Co-worker explains what he means and asks: *"When do you think it will be ready?"*
Pete is pointing to the screen: *"I'll take care of it when we're done here."*
Co-worker: *"Ok."* Then he leaves.
Sandra is still typing: *"We do it like this, alright?"* No reaction from Pete, Sandra continues typing in silence. After 40 seconds Sandra stops driving and asks: *"Can you continue?"*
Pete is taking the mouse and keyboard without starting to type: *"I'm completely lost at the moment. Why does he need to bother me with this right now?"*

This excerpt illustrates that Pete and Sandra were focusing together on the task until Pete is interrupted by a co-worker. He replies to the co-worker while Sandra keeps working on the task and does not respond to Sandra's questions. When she asks him to drive he makes an explicit statement that he does not know what Sandra is working on.

During the interviews, developers explained that interruptions are problematic because they disturb the developers in their process of thinking together about the problem. Developers are forced to think about another problem and this leads to disengagement with the task at hand.

"Interruptions do not work for me when I'm working on a tricky topic and thinking together with someone else. We just managed to connect our brains and then someone walks in and starts talking to us. We then have to think about their problem and I'm out again."

Interruptions initiated by a third party lead to developers disengaging from the task at hand because they have to focus on another problem.

B. Division of Work according to Expertise

We found that some developers agreed on a division of a task into subtasks; while one developer is solving a subtask the other one is disengaged. We observed this behaviour in pairs where the developers had complementary expertise.

This is illustrated with an excerpt of a session in which Alex and David are working on debugging a Java/C++ interface. In the questionnaires, they indicate that they solve that task as a pair because it requires different areas of expertise (C++ and Java). Alex's expertise is in C++, David's is in Java.

The pair tries to locate the problem. Alex is driving and David is suggesting an approach to address the problem within the C++ code.

Alex: *"No, that won't work. It crashes before it gets to the C++ code."*
David: *"It's not getting to the C++ code?"*
Alex: *"The first thing..."*
David interrupts: *"That means that it's quite likely that it is not a C++ problem but a Java one."*
Alex: *"That here <pointing with the mouse> is the constructor and we do not get to the MessageBox" <points to the MessageBox inside the constructor>.*
David: *"Where do we call that constructor?"*
Alex highlights something with the mouse on the screen.
David: *"And that is called in Java. So let's debug it from the Java side because everything else does not make sense."*
Alex: *"But I do not know the Java part."*
David is taking the keyboard: *"That does not matter. So I take the lead now."*
David starts driving and an episode of disengagement of Alex starts. Later in the session, they face a C++ task while David is still driving. He shifts the keyboard to Alex and says: *"Ok, I think I let you do that! Ok?"*
Alex takes the keyboard and starts typing.

This excerpt shows that David and Alex are mutually engaged in solving the problem till they face a subtask that is clearly in the field of David's expertise. In this case, David takes the lead and Alex is not engaged in this particular subtask. They take turns in driving and solving the task according to their expertise. They communicate that clearly with each other.

During the interviews, these developers stated that they do not know each others' area very well and that it is less time-consuming when the expert is solving the respective subtask. They perceived the PP session as positive and successful for the reason that they would not have been able to solve the task all by themselves. They did not perceive the disengagement of their partners during those subtasks as negative. It is important to note that according to the questionnaires and interviews, developers used PP in this pair constellation to solve a task that required different expertise. They did not focus on knowledge transfer nor were they interested into turning the C++ developer into a Java developer or vice versa.

Based on our analysis, disengagement due to division of work can occur when developers with complementary

expertise are working together. Both developers are engaged in the task as a whole, but subtasks requiring specific expertise are solved by one developer.

C. Simple Tasks

Disengagement can also occur when one of the developers is working on a simple subtask. In the interviews, developers reported that a simple task with low complexity does not need to be solved by two developers.

Excerpt from an interview where a developer talks to his partner about the refactoring task:
"[...] the part where the refactoring started. You could have done that without me. I was just sitting next to you in this situation. There was nothing to think about. [...] We did not build anything new [in this situation]. That was just about executing the task."

In the interviews, developers stated that they would consider splitting up the pair constellation for simple tasks that take long to solve. We found that developers who were solving the task did not mind that the other developer was not contributing to that task.

D. Social Pressure

We identified episodes of disengagement in PP sessions where an inexperienced pair programmer (with less than six months' PP experience [24]) paired with an experienced pair programmer. In these sessions, the inexperienced pair programmers were less knowledgeable than the experienced pair programmers.

The description of "less knowledgeable" is driven by the developers' perception of their own and their partner's knowledge. Here we are referring to the knowledge that is required to solve the given task and it is put in relation to their partner's knowledge. If both developers in a PP session agree that one partner has less knowledge to solve the task at hand, that partner is described as less knowledgeable. This means that our description of less knowledgeable is contextually based on the partners' knowledge and the specific task they are solving at the time.

In this paper we will refer to the inexperienced pair programmers who were less knowledgeable as "novice" and to the more knowledgeable and experienced pair programmers as "expert".

According to the questionnaire, one of the main reasons why developers worked in expert-novice constellations was to transfer knowledge from the expert to the novice. In some of those sessions, we observed that the novices were disengaged during the PP sessions; they avoided driving and they stopped asking and replying to questions. There was no specific event in the PP sessions that could explain these behaviours. In the interviews, novices expressed that they felt under pressure.

Excerpt 1: Tom (novice) and Max (expert)
Tom (novice): *"I feel like I have to understand it [explanations by his partner] as quickly as possible. So, that we can move on to solving the task and that we get started and"*.

Max (expert) interrupts: *"In order to not look stupid?"*
Tom (novice): *"Yes."*

Excerpt 2: Paul (novice) and Sebastian (expert)
Sebastian : *"I sometimes had the feeling that he was under a bit of pressure. Like he thought that I had expectations about how he solves the task. For example, he was writing a method and made lots of typos and he kept saying: I'm almost there."*
Paul (novice): *"It is odd when someone is sitting next to you and watches your hands while you're typing. And then you do not always know what to type."*

Excerpt 3: novice
"I'm really sorry that he has to pair with me. It is good for me because he is explaining very well but I know that I slow him down so much. He was driving most of the time but that is good. It is much faster when he is doing the typing. He knows exactly what to do but I'd need a lot of explanations."

The excerpts above show that less experienced developers can be worried about slowing their partners down or that they may look stupid. This can explain why they avoid driving and stop asking questions. We found that they stopped asking for explanations even if they did not understand what their partner was working on. This in turn, explains why they were not able to reply to questions regarding the task at hand.

E. Time Pressure

When we analysed these same novice-expert pairs from the perspective of the expert, we found that some experts were under a different type of pressure.

"Of course, it is slower when he is driving because they have to think about it [the problem] even more. But that is normal because they are not that familiar with the topic yet. That is no criticism. That is how it is when you're learning. But we are under deadline pressure at the moment. It was a bit stressful, so I just took the keyboard often. I had a look at the watch and thought we have to get that done on time. Otherwise I would have let him drive more often."

"If we would have had more time and not that much deadline pressure I would have explained the topic to him properly. [...] but we haven't had the time for that. However, I did not work on my own either so it was neither one thing nor another. I could not really explain it nor just try things quickly for myself."

Those interview excerpts illustrate that the experts were under time pressure and decided to focus on quick problem solving rather than on knowledge transfer. However, not all experts seem to be aware of knowledge transfer as the main motivation for using PP. They seem to try to solve the problem more quickly by taking control over mouse and keyboard and by cutting back on their explanations. These excerpts also show that the expert believes that they would work differently if they had more time.

Based on our analysis, we found two different types of behaviours that explain why developers disengage in expert-novice constellations. Novices might stop contributing despite the fact that they cannot follow their partners due to social pressure. Furthermore, we found that developers under

time pressure set higher priority on solving the task than on knowledge transfer. This happens even though developers stated before the session that the main reason to pair in this constellation is to achieve knowledge transfer. As a consequence, novices drop out of the PP session and can neither contribute to the task nor follow their partners' explanations or activities. In those cases, disengagement is unintentional and is not agreed. These two different types of pressure seem to be independent. Novices mentioned that they felt that they are under social pressure even when the experts did not state that they were under time pressure.

VII. FINDINGS: INTERACTIONS TO AVOID DISENGAGEMENT

In this section, we focus on disengagement due to social pressure or time pressure as it is unintentional and not agreed to between the developers.

This type of disengagement was exhibited in PP sessions where a novice paired with an expert. Based on the questionnaires, we found that knowledge transfer is one of the main reasons to pair in those constellations. As described in section VI, we found that in those constellations the novice faced the problem of dropping out.

Here, we present interactions between developers who did not suffer from disengagement issues despite working under the same circumstances. Our findings are illustrated with concrete examples from the video and interview data.

A. Encouraging the Novice to Drive

We observed scenarios where the expert encouraged their partner to drive. This is illustrated with a PP session in which Sara and George are working together. Sara is the novice and is driving. George is explaining to her something about debugging.

George: [...] *I usually work in the debug mode. It's not that really slower and I can set breakpoints whenever I need it.*

Sara: *Okay. So what is our next step?*

George: *"I wanted to show you that [task that they talked about before]."*

Sara puts takes her hands off the keyboard: *"Okay."*

George: *"No, no you can keep driving."*

Sara puts her hands back on the keyboard.

George replies: *"Now, you can work on the GUI again."*

Sara: *"I have no idea about that."*

Sara keeps driving and George starts explaining.

This excerpt shows that George encourages Sara to keep driving. Instead of taking the keyboard, he explains to her how to continue. It seems that developers who encouraged their partner to drive were aware of the fact that letting their partner drive might be more time-consuming but they set higher priority on knowledge transfer than on solving the task as quickly as possible. In one pair, developers decided even before the session that the novice would be driving. In the interview, the expert explained this strategy to his partner:

"Otherwise [if I were driving] you would have been lost very quickly because I'm just typing, typing, typing. [...] And that [you being driver] was the reason why the session took a bit longer. You were a bit nervous in the beginning. Maybe because you were a bit under pressure that you have to understand everything now."

The less experienced developers stated that they can work at their own pace when driving. This means they do understand the task and the problem solving activities and might not drop out.

Based on our analysis, we suggest that one strategy to avoid disengagement and in turn dropping out is to let the novice drive. Based on our observations, novices do not tend to take the initiative to be the driver and may need to be encouraged by their partners.

B. Verbalisation, Explanations and Feedback Questions

We found that some of the experts verbalised their steps while driving, explained their thoughts and asked actively for feedback from their partners.

Stephen is driving the whole time during this excerpt. He is writing a comment in the code and verbalises what he is writing: *"This is needed for the test [...] we identify them based on their email subject. <he finishes writing his comment> Okay?"*

Jen: *"Okay. And no one looks for typos."*

Stephen opens the tab with the test: *"Okay. Here is the test. GetEmailContent. This requires the calculation <calculation as input for the method> so this would be our subject."*

He types *"String subject"*.

Jen: *"A string. Subject getEmailContent, the subject was the first entry, wasn't it?"*

Stephen types *"[0]"*.

Jen: *"Or the zeroth entry in an array."*

Stephen: *"I will just have a look whether there is already an activityService here. < he is browsing through the code> "There is one but that is the wrong place, I think."*

This excerpt illustrates that Stephen is verbalising while typing and he asks for feedback from Jen. Additionally, he is making his thoughts visible to Jen when looking for the *"activityService"*.

We suggest that verbalisation, explanation and feedback questions can avoid disengagement and in turn dropping out. Even if the novice stops asking questions due to time pressure or social pressure, their partners' verbalisation and explanation might provide enough insights into their activities and thinking so that they could still follow their partner.

C. Asking for Clarification

We observed that novices ask for clarification when they could not follow their partners or to make sure that they understood the explanation correctly.

Andreas is driving the whole time during this excerpt.

Andreas: *"So, that is the createFolder method."*

Philip: *"Right, so let's go to the test."*

Andreas opens the tab with the test: *"So, let's have a look at the test. Now we can..."*

Philip interrupts: *"Oh, wait, I have to understand that first."*

Andreas: “Okay, wait a second, I just write the comment here and then I explain it to you. Just one second.”

Andreas writes the comment and starts explaining.

Philip is the novice in this constellation. He stops Andreas when he realises that he does not understand the code in the test and asks for clarification. Philip tells him to wait for a moment and then starts his explanation. This shows that Philip makes sure that he understands the code that is required to solve the problem. We also observed that novices repeat the explanation of their partner in their own words and ask subsequently for feedback. This is another strategy to make sure that they understood their partners’ explanation or activities correctly.

Asking for clarification can avoid disengagement in terms of dropping out because the novice makes sure that s/he can follow their partner at anytime. This means the novice take the initiative and stop their partner when clarification of their actions is required.

VIII. DISCUSSION

A. Acceptable Disengagement

We describe disengagement as acceptable when the developers did not mind that their partners disengaged. There are two sets of circumstances where developers perceived episodes of disengagement as acceptable: when solving a simple tasks that does not require input from both developers (presented in section VI-C), and when developers had complementary knowledge and developers solved subtasks according to their expertise (presented in section VI-B).

Developers agreed that solving a simple task does not need input from both developers. The developers who disengaged stated that there was no need for them to contribute and that they would split up the pair constellation for simple tasks that are time-consuming. This is consistent with the findings from other studies [6], [25] that reported that pairing was not found useful in simple tasks. However, Hulkko and Abrahamsson [26] found that pairing helped to find mistakes in simple tasks as “the coder himself might have become blind” to those mistakes.

Pairing of developers with complementary knowledge was motivated by the fact that the task at hand required different areas of expertise. The main motivation was not to exchange expertise. This means that disengagement during certain subtasks was not a threat to the goal of their PP session, i.e. solving a task that requires different areas of expertise.

In the circumstances described, disengagement is not necessarily harmful for PP. However, this depends on the motivation for using PP; being disengaged because the partner does not need input while solving a subtask might not be a threat to the goal of their PP session but the pair loses the effect of pair reviews for those subtasks and the effect of pair learning. This emphasises the importance of being aware of the motivation for using PP.

B. Harmful Disengagement due to Conflicting Goals

In some expert-novice constellations, novices dropped out of their PP sessions. Dropping out means that novices are not able to follow their partner’s explanations or activities.

Those constellations were mainly motivated by the idea that the expert transfers knowledge to the novice. Dropping out undermines knowledge transfer since the novice is not able to follow their partner’s explanation. Therefore, we consider dropping out as harmful to PP.

Our findings show that in those situations, experts took the lead in terms of driving and they felt that they did not explain enough to the novice. According to the developers, their behaviours were due to time pressure. This means that developers had to face conflicting goals in this session: time pressure vs. knowledge transfer.

Knowledge transfer takes time and hence is not compatible with time pressure. Time pressure can hinder establishing a sufficiently uniform level of expertise [20] as trying to “educate” a less competent partner takes extra time and effort [27].

Another factor is social pressure. Novices avoided driving and stopped asking for explanation due to social pressure even though they could not follow their partners’ activities and explanations. They were facing a conflict as well. They had to overcome the social pressure in order to achieve learning goals. However, based on our observations, it seems that some developers were not able to overcome the social pressure and dropped out of the session. As discussed above, this emphasizes the importance of being aware of the motivation for using PP for a specific task and moreover the importance to create an environment that allows the developers to achieve their goals and support them to overcome these conflicts. We discuss strategies for this in section VIII-D.

Another factor that could influence the novices’ behaviours and lead to dropping out is the difficulty of the task. Vygotsky [28] refers to the “zone of proximal development” in which learners are able to solve problems that they cannot solve independently, in collaboration with more capable peers. Murray and Arroyo [29] point out that overwhelming learners with difficult tasks which are out of their “zone of proximal development” can confuse learners and “lead to distraction, frustration and lack of motivation”. In our study developers did not state that they were overwhelmed with the difficulty of the task. However, frustration and a lack of motivation could lead to dropping out.

C. Visible vs Invisible Disengagement

Disengagement is not always visible. The observable behaviours of developers can only indicate disengagement. In some sessions, developers verbally made explicit to their partners that they could not follow them or did not understand what they were working on. In other cases, developers did not make their disengagement visible to their

partners. Whenever disengagement is agreed, for example, for solving simple task or for division of subtasks, the visibility and awareness of disengagement is not important. In those circumstances developers do not mind when their partner disengages.

However, the visibility of disengagement is important when developers drop out of their PP sessions. Statements in the interviews revealed that developers were not necessarily aware that their partner had dropped out. If developers could tell that their partners have dropped out then they could, for example, go a few steps back and explain the task again. However, this requires awareness of disengagement.

In the context of computer supported cooperative work, Dourish and Bellotti [30] emphasise the role of awareness for successful collaboration; “awareness is fundamental to coordination of activities and sharing of information, which, in turn, are critical to successful collaboration.”

Visibility of disengagement is important as it allows awareness and uncovers the need for coordination activities and information sharing to get the partner back on track.

D. Minimizing the Risk of Harmful Disengagement

Developers show behaviours that lead to harmful disengagement because of social pressure and time pressure.

Time pressure is a pressure that does not arise within the pair instead it arises due to the circumstances within which the PP session takes place. Addressing the issue of time pressure might help the experts to set high priority on knowledge transfer. This may also reduce the social pressure on the novice as they may have been worried about slowing their partner down. We suggest that developers should be aware that sessions for which knowledge transfer is the primary goal should not take place under time pressure. This has implications for the planning of these sessions. We propose that time estimations for tasks should be adapted depending on whether the task will be solved in a pair and furthermore in which PP constellation it will be solved. Expert-novice constellations might need longer for solving a task than expert-expert constellations.

In contrast to time pressure, social pressure is an individual pressure. We observed that some novices are proactive and ask the expert for clarifications when they are needed. Novices might have overcome their social pressure or might not be susceptible to that type of pressure. If a novice is susceptible to social pressure and can't overcome it we suggest two strategies that might avoid disengagement of the novice: (1) the expert can encourage the novice to drive. The novice will work at their own pace rather than struggling to keep up with the expert. Additionally, the novice has to understand the task and the approaches to solve the problem in order to do the driving. (2) Experts can prompt feedback questions, verbalise what they were working on and explain their thoughts while working. This could help the novice to

understand the task at hand and the reasoning behind the experts' interactions. Making thinking visible can improve the understanding of “reasoning and strategies that experts employ when they acquire knowledge or put it to work to solve complex or real-life tasks” [31]. Furthermore, this behaviour might be beneficial for the expert as well because simply talking about the task may support its understanding and eventually its resolution [18].

IX. SUMMARY AND CONCLUSION

This paper reports on an empirical study that investigates disengagement in industrial pair programming sessions. Our findings suggest that disengagement is sometimes acceptable and agreed upon between the developers. However, we also found harmful episodes of disengagement in expert-novice constellations; novices drop out of their PP sessions and are not able to follow their partner's work nor contribute to the task at hand, thus losing the expected benefits of knowledge transfer. This can be addressed by the experts encouraging the novice to drive and by providing sufficient explanation while working. This assumes however, that the expert is aware that the novice has dropped out. This is not necessarily the case because disengagement may not always be visible and the expert, focusing on the task at hand, may not notice. Additionally, experts might have to deal with time pressure. Time pressure is conflicting with the motivation to achieve knowledge transfer. In order to minimize harmful disengagement, developers should be aware of their motivation for a given PP session and create an environment that allows them to achieve their goals. Developers should also maintain awareness about the state of engagement of their partners. This will allow them to achieve their goals for a given PP session by helping each other stay mutually engaged.

ACKNOWLEDGMENT

The authors would like to thank the participating companies.

REFERENCES

- [1] J. Stephen, J. Page, J. Myers, A. Brown, D. Watson, and I. Magee, “System error fixing the flaws in government it,” Institute for Government, Tech. Rep., 2011.
- [2] J. Vanhanen and C. Lassenius, “Perceived effects of pair programming in an industrial context,” in *Conference on Software Engineering and Advanced Applications*. IEEE Computer Society, 2007, pp. 211–218.
- [3] L. Layman, L. Williams, and L. Cunningham, “Exploring extreme programming in context: an industrial case study,” in *Agile Development Conference*, 2004, pp. 32–41.
- [4] J. Vanhanen and H. Korpi, “Experiences of using pair programming in an agile project,” in *International Conference on System Sciences*. IEEE Computer Society, 2007, p. 274b.

- [5] A. Begel and N. Nagappan, "Pair programming: what's in it for me?" in *International symposium on Empirical software engineering and measurement*. ACM, 2008, pp. 120–128.
- [6] L. Williams, R. Kessler, W. Cunningham, and R. Jeffries, "Strengthening the case for pair programming," *IEEE software*, vol. 17, no. 4, pp. 19–25, 2000.
- [7] J. Hannay, T. Dybå, E. Arisholm, and D. Sjøberg, "The effectiveness of pair programming: A meta-analysis," *Information and Software Technology*, vol. 51, no. 7, pp. 1110–1122, 2009.
- [8] E. Arisholm, H. Gallis, T. Dybå, and D. I.K. Sjøberg, "Evaluating pair programming with respect to system complexity and programmer expertise," *IEEE Transactions on Software Engineering*, vol. 33, no. 2, pp. 65–86, 2007.
- [9] J. E. Hannay, E. Arisholm, H. Engvik, and D. I. Sjøberg, "Effects of personality on pair programming," *IEEE Transactions on Software Engineering*, vol. 36, no. 1, pp. 61–80, 2010.
- [10] L. Williams and R. Kessler, *Pair programming illuminated*. Addison-Wesley Longman Publishing Co., Inc. Boston, MA, USA, 2002.
- [11] S. Wray, "How pair programming really works," *IEEE Software*, vol. 27, no. 1, 2009.
- [12] J. Roschelle and S. Teasley, "The construction of shared knowledge in collaborative problem solving," *NATO ASI Series / Computer and Systems Sciences*, vol. 128, pp. 69–69, 1994.
- [13] L. Plonka, H. Sharp, and J. van der Linden, "Investigating equity of participation in pair programming," in *Agile India: International conference on agile and lean software methods*. IEEE Computer Society, 2012.
- [14] L. Lipponen, "Exploring foundations for computer-supported collaborative learning," in *Conference on Computer Support for Collaborative Learning: Foundations for a CSCL Community*. International Society of the Learning Sciences, 2002, pp. 72–81.
- [15] D. Miell and R. MacDonald, "Children's creative collaborations: The importance of friendship when working together on a musical composition," *Social Development*, vol. 9, no. 3, pp. 348–369, 2000.
- [16] N. Bryan-Kinns, P. Healey, and J. Leach, "Exploring mutual engagement in creative collaborations," in *conference on Creativity & cognition*. ACM, 2007, pp. 223–232.
- [17] T. Walle and J. Hannay, "Personality and the nature of collaboration in pair programming," in *International Symposium on Empirical Software Engineering and Measurement*. IEEE Computer Society, 2009, pp. 203–213.
- [18] S. Bryant, P. Romero, and B. Boulay, "The collaborative nature of pair programming," in *Proceedings of Extreme Programming and Agile Processes in Software Engineering*. Springer, 2006, pp. 53–64.
- [19] S. Bryant, "Double trouble: Mixing qualitative and quantitative methods in the study of extreme programmers," in *Symposium on Visual Languages and Human Centric Computing*, 2004, pp. 55–61.
- [20] J. Chong and T. Hurlbutt, "The social dynamics of pair programming," in *International Conference on Software Engineering*, 2007, pp. 354–363.
- [21] M. Rostaher and M. Hericko, "Tracking test first pair programming - an experiment," in *Proceedings of the Second XP Universe and First Agile Universe Conference on Extreme Programming and Agile Methods*. Springer, 2002, pp. 174–184.
- [22] S. Bryant, P. Romero, and B. du Boulay, "Pair programming and the mysterious role of the navigator," *International Journal of Human Computer Studies*, vol. 66, no. 7, pp. 519–529, 2008.
- [23] L. Plonka, J. Segal, H. Sharp, and J. van der Linden, "Collaboration in pair programming: driving and switching," in *International Conference on Agile Processes in Software Engineering and Extreme Programming*. Springer, 2011, pp. 43–59.
- [24] S. Freudenberg, "The 'tag team': Tools, tasks and roles in collaborative software development," Ph.D. dissertation, University of Sussex, 2006.
- [25] M. Müller and W. Tichy, "Case study: extreme programming in a university environment," *International conference on Software engineering*, pp. 537–544, 2001.
- [26] H. Hulkko and P. Abrahamsson, "A multiple case study on the impact of pair programming on product quality," in *International conference on Software engineering*. ACM, 2005, pp. 495–504.
- [27] L. Cao and P. Xu, "Activity patterns of pair programming," in *International Conference on System Sciences*, 2005, pp. 88a–88a.
- [28] L. Vygotsky, *Mind in Society: The development of higher psychology processes*. Cambridge MA: Harvard University press, 1978.
- [29] T. Murray and I. Arroyo, "Toward measuring and maintaining the zone of proximal development in adaptive instructional systems," in *International Conference on Intelligent Tutoring Systems*. Springer, 2002, pp. 749–758.
- [30] P. Dourish and V. Bellotti, "Awareness and coordination in shared workspaces," in *Conference on Computer-supported cooperative work*. ACM, 1992, pp. 107–114.
- [31] A. Collins, J. Brown, and A. Holum, "Cognitive apprenticeship: Making thinking visible," *American Educator*, vol. 15, no. 3, pp. 6–11, 1991.